

Docker Disk Cleanup — DroneArjuna Dev Environment

Audience: DroneArjuna developers on Windows + Docker Desktop (WSL2 backend)
Purpose: Reclaim C: drive space consumed by Docker, safely, without losing dev data unless intended.
Date: 2026-08-05

1. Why Docker eats your C: drive

On Windows, Docker Desktop runs containers inside a WSL2 virtual machine. All images, containers, volumes, and build cache live inside virtual disk files (`.vhdx`), not as normal files on C:. Two problems follow from this:

- 1. Deleting Docker objects (images, containers) frees space *inside* the `vhdx`, but Windows does **not** automatically shrink the `vhdx` file itself — the space isn't returned to C: until you compact it.
- 2. Docker's build cache and dangling image layers accumulate silently across `docker compose up --build` runs and are easy to forget about.

Snapshot from this machine (2026-08-05):

File	Purpose	Size on disk
<code>...\Docker\wsl\disk\docker_data.vhdx</code>	Main Docker data disk (images, containers, volumes, build cache)	15.51 GB
<code>...\Docker\wsl\main\ext4.vhdx</code>	WSL2 system disk	0.1 GB

Actual DroneArjuna volume usage (`docker system df -v`):

Volume	Size
<code>dronearjuna_postgres_data</code>	113.4 MB
<code>dronearjuna_timescale_data</code>	73.58 MB
<code>dronearjuna_rabbitmq_data</code>	103.2 kB
<code>dronearjuna_es_data</code>	53.36 kB
<code>dronearjuna_pgadmin_data</code>	167.9 kB
<code>dronearjuna_minio_data</code>	14.15 kB
<code>dronearjuna_redis_data</code>	89 B

~187 MB of real data inside a 15.51 GB file — the remaining ~15.3 GB is reclaimable slack (old image layers, build cache, orphaned data from prior builds/prunes that was freed logically but never compacted physically).

2. Decision guide — which level of cleanup do I need?

Situation	Use
Just want to tidy up, keep dev DB data	§3 Safe cleanup
Rebuilt images many times, disk usage climbing	§3 + §5 (compact vhdx)
Want a clean slate, don't care about dev DB/queue data	§4 Destructive cleanup
C: drive nearly full, need space back <i>now</i>	§5 Compact vhdx (do this regardless of §3/§4)

3. Safe cleanup (no data loss)

Check usage first:

```
docker system df
docker system df -v      # verbose, per-image/volume breakdown
```

Then clean non-destructive targets:

```
# Dangling images (untagged layers left over from rebuilds)
docker image prune

# Unused build cache
docker builder prune

# Stopped containers
docker container prune

# All of the above at once, excluding volumes and tagged-but-unused images
docker system prune
```

None of these touch named volumes (`dronearjuna_postgres_data` , etc.) or images still tagged/in-use, so your Postgres/TimescaleDB/Redis/RabbitMQ/MinIO dev data is untouched.

4. Destructive cleanup (⚠ confirm before running)

Removes **all** unused images (even tagged ones not currently referenced) and **all unused volumes system-wide** — not just DroneArjuna's.

```
docker system prune -a --volumes
```

Project-scoped equivalent — wipes DroneArjuna's own containers + named volumes only:

```
docker compose down -v
```

This destroys `postgres-data` , `timescale-data` , `redis-data` , `rabbitmq-data` , `minio-data` , `es-data` , `pgadmin-data` for this project. You will need to re-run migrations (`alembic upgrade head`) and re-seed data

afterward.

Per CLAUDE.md working-style rule: always confirm before `docker compose down -v` or any volume-destroying command.

5. Compact the vhdx (reclaim space on C: itself)

Steps 3/4 free space *inside* the virtual disk but don't shrink the file. To get physical space back on C:, do this:

1. **Quit Docker Desktop completely** (system tray → Quit, not just close window).
2. Shut down WSL:

```
wsl --shutdown
```

3. Locate your vhdx (already found on this machine — path may differ per install):

```
C:\Users\<YourUser>\AppData\Local\Docker\wsl\disk\docker_data.vhdx
```

4. Compact it via `diskpart` :

```
diskpart
```

Inside the diskpart prompt:

```
select vdisk file="C:\Users\<YourUser>\AppData\Local\Docker\wsl\disk\docker_data.vhdx"
attach vdisk readonly
compact vdisk
detach vdisk
exit
```

5. Restart Docker Desktop and verify the stack still comes up:

```
docker compose up -d
docker compose ps
```

Easier alternative (recent Docker Desktop versions): Settings → Resources → Advanced → "Clean / Purge data", or Settings → Resources → WSL Integration has a disk compaction option built in. Check your Docker Desktop version before doing the manual diskpart route.

6. Recommended routine for this project

Given DroneArjuna's dev data is small (~187 MB) and cheap to recreate (Alembic migrations, no large seed datasets noted), a reasonable cadence:

- **Weekly / after heavy rebuild sessions:** `docker system prune` (safe).

- **Monthly, or when C: is tight:** `wsl --shutdown` + `diskpart compact ($5)` — safe, no data loss, since it only compacts free space.
- **Only when starting a fresh Phase or debugging a corrupted DB state:** `docker compose down -v` (destructive, confirm first).

7. Quick reference

```
# Check usage
docker system df -v

# Safe cleanup
docker system prune

# Destructive – DroneArjuna project only
docker compose down -v

# Destructive – system-wide, all images + volumes
docker system prune -a --volumes

# Reclaim C: drive space (after cleanup)
wsl --shutdown
diskpart
# select vdisk file="C:\Users\<user>\AppData\Local\Docker\wsl\disk\docker_data.vhdx"
# attach vdisk readonly
# compact vdisk
# detach vdisk
# exit
```